

Modern JavaScript

React Öncesi Gerekli JS Bilgisi

Session Akışı

Süre	Konu	Anahtar Kavramlar
0-20 dk	Değişkenler ve Fonksiyonlar	let/const, arrow function, template literal
20-50 dk	Veri Dönüştürme	destructuring, spread/rest, map/filter/reduce
50-75 dk	Asenkron JavaScript	Promise, async/await, fetch API
75-90 dk	Pratik: API'den liste çek	fetch + render

1 JS Değişkenler ve Modern Fonksiyonlar

🕒 0–20 dakika

1.1 var, let, const Farkı

JavaScript'te değişken tanımlamak için üç anahtar kelime vardır. Modern kodda **var kullanılmaz**; **let** ve **const** tercih edilir.

var vs let vs const

```
// var - avoid, can cause problems
var name = "Ahmet";
var name = "Mehmet"; // redeclaration: no error (bad)

// let - mutable value
let score = 0;
score = 10;          // OK

// const - constant binding (for primitives the value is
// constant)
const PI = 3.14;
// PI = 3;           // ERROR: const cannot be reassigned

// const with object/array: binding is constant, contents can
// change
const user = { name: "Ali", age: 30 };
user.age = 31; // OK - content changed
// user = {};  // ERROR - binding cannot change
```

	Yeniden atama	Yeniden tanımlama	Kapsam
var	Evet	Evet	Fonksiyon
let	Evet	Hayır	Blok ({ })
const	Hayır	Hayır	Blok ({ })

💡 Altın Kural

Önce const dene. Değer değişmesi gerekiyorsa **let** kullan. **var**'a hiç gerek yok.

1.2 Arrow Functions (Ok Fonksiyonları)

Arrow function, fonksiyon yazmanın daha kısa yoludur. React'ta her yerde karşımıza çıkacak.

Normal vs Arrow Function

```
// --- Normal function ---
function sum(a, b) {
  return a + b;
}

// --- Arrow function (equivalent) ---
const sum = (a, b) => {
  return a + b;
};

// --- Short form: single expression => no {} and no return ---
const sum = (a, b) => a + b;

// --- Single parameter: parentheses optional ---
const square = x => x * x;

// --- No parameters ---
const greet = () => "Hello!";

// Usage
console.log(sum(3, 5)); // 8
console.log(square(4)); // 16
console.log(greet()); // "Hello!"
```

💡 Ne Zaman Kullanılır?

React'ta component tanımlarında, event handler'larda ve map/filter gibi call-back'lerde arrow function kullanmak standart haline gelmiştir.

1.3 Template Literals (Şablon Dizeler)

Değişken birleşimi için + yerine daha okunaklı bir yöntem.

Template Literal Örneği

```
const name = "Zeynep";
const age  = 28;

// Old way - harder to read
const message1 = "Hello, " + name + "! Your age: " + age;

// Template literal - written with backticks (`)
const message2 = `Hello, ${name}! Your age: ${age}`;

// You can use expressions too
const message3 = `${age >= 18 ? "Adult" : "Minor"}: ${name}`;

// Multiline text
const html = `
  <div>
    <h1>${name}</h1>
    <p>Age: ${age}</p>
  </div>
`;
```

2 Veri Dönüştürme Araçları

 20–50 dakika

2.1 Destructuring (Parçalama)

Nesne veya diziden değerleri tek satırda çıkarmak için kullanılır. React'ta **props** ve **state** kullanımı buna dayanır.

Object Destructuring

```
const user = {
  name: "Can",
  age: 25,
  city: "Istanbul"
};

// Old way
const name = user.name;
const age = user.age;

// With destructuring
const { name, age } = user;

// Renaming
const { name: firstName, age: yearsOld } = user;
console.log(firstName); // "Can"

// Default value
const { country = "Turkey" } = user;
console.log(country); // "Turkey" (not present in
                      object)

// In function parameters — React props often look like this
function UserCard({ name, age }) {
  return `${name} (${age})`;
}
```

Array Destructuring

```
const colors = ["red", "green", "blue"];

// Destructuring
const [first, second] = colors;
console.log(first); // "red"
console.log(second); // "green"

// Skipping
const [, , third] = colors;
console.log(third); // "blue"

// React useState uses this exact format:
const [count, setCount] = useState(0);
//      ^value  ^updater
```

2.2 Spread ve Rest Operatörleri (...)

Üç nokta (...) iki farklı işlem için kullanılır: **spread** (yaymak) ve **rest** (toplamak).

Spread — Dizi ve Nesne Birleştirme

```
// Dizi spread
const a = [1, 2, 3];
const b = [4, 5, 6];
const all = [...a, ...b];           // [1, 2, 3, 4, 5, 6]
const copy = [...a];                // does not affect original

// Object spread — React state updates often look like this
const user = { name: "Ali", age: 30 };
const updated = { ...user, age: 31 };
// { name: "Ali", age: 31 }

// Fonksiyon çağrılarında
const numbers = [3, 1, 4, 1, 5];
console.log(Math.max(...numbers)); // 5
```

Rest — Kalan Parametreleri Topla

```
// Fonksiyon parametrelerinde rest
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10

// Rest in destructuring
const [first, second, ...rest] = [10, 20, 30, 40, 50];
console.log(first);           // 10
console.log(rest);            // [30, 40, 50]

const { name, ...otherInfo } = user;
// otherInfo = { age: 30 }
```

2.3 Array Methods — map, filter, reduce

Neden Önemli?

React'ta liste render etmek için `map`, veri filtrelemek için `filter` kullanılır. Bu metotlar orijinal diziyi **değiştirmez, yeni dizi döndürür**.

2.3.1 map — Her Elemanı Dönüştür

map Örnekleri

```
const numbers = [1, 2, 3, 4, 5];

// Square each number
const squares = numbers.map(n => n * n);
// [1, 4, 9, 16, 25]

// Transform an array of objects
const users = [
  { id: 1, name: "Ali" },
  { id: 2, name: "Veli" },
];

const names = users.map(u => u.name);
// ["Ali", "Veli"]

// In React: create a list of elements
// users.map(u => <li key={u.id}>{u.name}</li>)
```


2.3.2 filter — Koşulu Sağlayanları Al

filter Örnekleri

```
const products = [
  { name: "Pencil", price: 5, inStock: true },
  { name: "Notebook", price: 15, inStock: false },
  { name: "Eraser", price: 3, inStock: true },
  { name: "Ruler", price: 8, inStock: true },
];

// In-stock products
const inStock = products.filter(p => p.inStock === true);
// [{Pencil}, {Eraser}, {Ruler}]

// Products cheaper than 10
const cheapOnes = products.filter(p => p.price < 10);
// [{Pencil}, {Eraser}, {Ruler}]

// Search filter example
const query = "pe";
const results = products.filter(p =>
  p.name.toLowerCase().includes(query)
);
// [{Pencil}]
```

2.3.3 reduce — Tek Değere İndir (Yüzeysel)

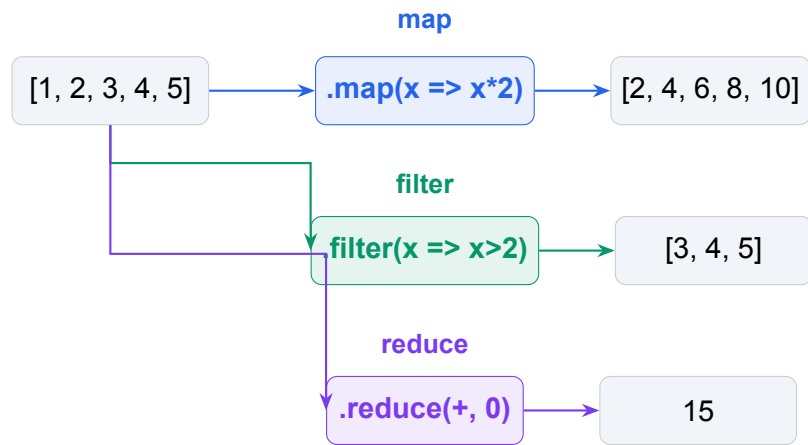
reduce Örneği

```
const prices = [10, 25, 8, 42, 15];

// Toplam hesapla
const total = prices.reduce((acc, price) => acc + price, 0);
//                               ^birikim ^su anki eleman ^
// baslangic
console.log(total); // 100

// Nesne dizisinde toplam
const cart = [
  { product: "Pencil", quantity: 3, price: 5 },
  { product: "Notebook", quantity: 1, price: 15 },
];

const totalAmount = cart.reduce(
  (acc, item) => acc + (item.quantity * item.price),
  0
);
console.log(totalAmount); // 30
```



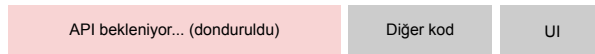
3 🕒 Asenkron JavaScript

🕒 50–75 dakika

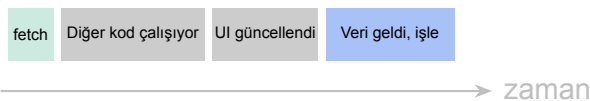
3.1 Neden Asenkron?

JavaScript **tek thread**'lidir: bir işlem bitene kadar diğer işlemler bekler. API'den veri çekme, dosya okuma gibi işlemler zaman alır. Bunları **asenkrone** yaparak kullanıcı arayüzü donup kalmasını önleriz.

Senkron (kötü):



Asenkron (iyi):



3.2 Promise

Promise, “*şu an elimde yok ama geldiğinde sana vereyim*” sözü veren bir nesnedir.

Promise Mantığı

```
// Promise has 3 states: pending -> fulfilled | rejected
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Data arrived!"); // fulfilled
  } else {
    reject("An error occurred!"); // rejected
  }
});

// Listen for the result
promise
  .then(data => console.log(data)) // "Data arrived!"
  .catch(err => console.error(err));

// fetch() returns a Promise:
fetch("https://api.example.com/users")
  .then(response => response.json()) // convert to JSON
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

3.3 async / await

async/await, Promise'leri **sanki senkronmuş gibi** yazmanın yoludur. Okunması ve hata yakalaması çok daha kolaydır.

async/await — Temel Kullanım

```
// An \"async\" function returns a Promise
async function fetchUsers() {
  // \"await\" waits for a Promise without freezing the UI
  thread

  const response = await fetch(
    \"https://jsonplaceholder.typicode.com/users\"
  );

  // response.json() also returns a Promise
  const data = await response.json();

  return data;
}

// Error handling: try/catch
async function fetchSafely() {
  try {
    const response = await fetch(\"https://api.example.com/data
      \");

    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }

    const data = await response.json();
    console.log(data);
  } catch (err) {
    console.error(\"An error occurred:\", err.message);
  }
}
```

.then().catch()

Zincir şeklinde yazılır
Her adımda callback
Bazı durumlarda daha kısa

async/await

Düz sıra gibi okunur
try/catch ile hata yakalar
Daha okunaklı, tercih edilir

3.4 fetch API

`fetch`, tarayıcının dahili HTTP istek fonksiyonudur. Dış kütüphanesiz API çağrısı yapmak için kullanılır.

fetch ile GET ve POST

```
// GET - veri çekme (en sık kullanılan)
async function postlariGetir() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts"
  );
  const postlar = await response.json();
  return postlar; // dizi donuyor
}

// POST - veri gönderme
async function yeniPostOlustur(baslik, icerik) {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ baslik, icerik }),
    }
  );
  const yeniPost = await response.json();
  console.log("Olusturuldu:", yeniPost);
}
```

Pratikte Kullanacağımız API

JSONPlaceholder — <https://jsonplaceholder.typicode.com>
Ücretsiz, kayıt gerektirmeyen sahte (fake) REST API. /posts, /users, /todos endpoint'leri var.

4 Pratik: API'den Veri Çekme ve Liste Render

 75–90 dakika

☰ Pratik Hedef

JSONPlaceholder API'sinden kullanıcı listesi çek ve sayfada listele.

- `fetch` ile `GET /users` isteği at
- Gelen kullanıcıları `map` ile HTML'e dönüştür
- Sayfaya enjekte et

Adım 1 — HTML Yapısı

index.html

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8" />
  <title>User List</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header>
    <h1>Users</h1>
  </header>

  <main>
    <!-- JS will render cards here -->
    <div id="user-list">
      <p>Loading...</p>
    </div>
  </main>

  <!-- Script at the end -->
  <script src="app.js"></script>
</body>
</html>
```

Adım 2 — JavaScript (app.js)

app.js

```
// 1. Helper that returns a user card HTML
function userCard(user) {
  return `
    <div class="card">
      <h2>${user.name}</h2>
      <p>${user.email}</p>
      <p>${user.address.city}</p>
    </div>
  `;
}

// 2. Fetch from API and render to the page
async function loadUsers() {
  const list = document.getElementById("user-list");

  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );

    if (!response.ok) throw new Error("API error");

    const users = await response.json();

    // Generate HTML with map, merge with join
    list.innerHTML = users
      .map(u => userCard(u))
      .join("");

  } catch (err) {
    list.innerHTML = `<p class="error">Error: ${err.message}</p>`;
  }
}

// Run on page load
loadUsers();
```

Adım 3 — CSS (style.css)

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: sans-serif;
  background: #f1f5f9;
  color: #1e293b;
}

header {
  background: #2563eb;
  color: white;
  padding: 24px;
  text-align: center;
}

#user-list {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  padding: 32px;
  justify-content: center;
}

.card {
  background: white;
  border-radius: 10px;
  padding: 20px;
  width: 240px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
}

.card h2 { font-size: 1rem; margin-bottom: 8px; color: #2563eb; }
.card p { font-size: 0.875rem; color: #64748b; margin-bottom: 4px; }

.error { color: #dc2626; }
```


5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ `const` / `let` farkı ve neden `var` kullanılmaz
- ✓ Arrow function kısaltmaları
- ✓ Template literal ile dinamik metin
- ✓ Object ve array destructuring
- ✓ Spread (`...`) ile kopyalama ve birleştirme
- ✓ `map` ile liste dönüştürme
- ✓ `filter` ile koşullu filtreleme
- ✓ `reduce` ile tek değere indirme
- ✓ Promise mantığı ve `async/await`
- ✓ `fetch` ile API'den veri çekme
- ✓ Pratik: API listesini sayfaya render etme

5.2 Sıkça Sorulan Sorular

SSS

S: Arrow function ile normal function her zaman birbirinin yerini alabilir mi?

C: Hemen hemen. Tek fark `this` bağlamı; method olarak tanınırken normal function tercih edilebilir, callback ve React component'lerinde arrow kullanılır.

S: `map` mi `forEach` mi?

C: `map` yeni dizi döndürür (React'ta bu gerekir). `forEach` sadece gezdirmek içindir, geriye hiçbir şey döndürmez.

S: `async` olmayan fonksiyonda `await` kullanılabilir mi?

C: Hayır. `await` yalnızca `async` olarak işaretlenmiş fonksiyon içinde kullanılabilir.

5.3 Sonraki Session — Session 3: React Temelleri

→ Session 3 (4 Mayıs Pazartesi)

Öğrendiğimiz JavaScript bilgisiyle React'a adım atıyoruz:

- React nedir, SPA (Single Page Application) mantığı
- **Component** yapısı ve **JSX** söz dizimi
- **Props** ile veri aktarımı
- **useState** ile durum yönetimi ve event handling